AST9 Health Hub — Phase 3+ Master Continuation Prompt

EXECUTIVE SUMMARY

Phase 2 is COMPLETE. You have a working platform with:

- Scoring Engine (ROM × Control × Force × Neurology)
- Gait Phase Analysis (7 phases, 14 rules)
- Program Generator (20 rules, warmups/cooldowns/daily routines)
- 3D Body Map (Three.js + SVG fallback)
- Dashboard with Supabase integration

Your job: Build Phase 3 features on top of this foundation. Do NOT rewrite Phase 2 code.

PHASE 3 SCOPE (4 Workstreams)

WORKSTREAM A: Community Features [HIGHEST PRIORITY]

Business Value: Differentiates AST9 from generic coaching apps. Enables coach collaboration and client retention through peer support.

Deliverables:

1. Coach-to-Coach messaging (peer network)
2. Client referral system between coaches
3. Case study sharing board (anonymized)
4. Client community: progress sharing, Q&A, support groups
5. Privacy controls (client controls who sees what)

Files to Create:

- `js/community.js` — all community logic
- `js/communityUI.js` — DOM rendering for community sections
- SQL migration for 10 new tables

Files to Modify:

- `index.html` — add community sections to sidebar + main content
- `dashboard.js` — add community notification badges, referral counts
- `css/styles.css` — community-specific styles

WORKSTREAM B: Exercise Library Integration

Business Value: Replaces hardcoded exercise strings with searchable, video-backed catalog.

Deliverables:

1. YouTube Data API v3 integration (playlist import)

2. Exercise database with video embeds
3. Search/filter by phase, category, body part
4. "Add to Program" button that feeds ProgramGenerator
5. Auto-populate from YouTube playlists per phase

Files to Create:

- `js/exerciseLibrary.js` — API calls, search, filter logic
- `js/exerciseUI.js` — grid rendering, video embeds

Files to Modify:

- `js/programGenerator.js` — replace hardcoded exercise strings with library lookups
- `index.html` — replace "Coming Soon" Exercise Library section
- `css/styles.css` — video card styles

WORKSTREAM C: Progress Visualization

Business Value: Coaches can show clients tangible progress = retention + referrals.

Deliverables:

1. Score history line charts (composite + 4 dimensions over time)
2. Radar chart showing current vs previous assessment
3. Gait phase improvement timeline
4. Dashboard sparklines in client cards
5. Exportable PDF progress report

Files to Create:

- `js/charts.js` — Chart.js wrappers for AST9 data
- `js/progressReport.js` — PDF generation logic

Files to Modify:

- `dashboard.js` — add chart initialization calls
- `index.html` — add chart containers to dashboard + client detail view

---

WORKSTREAM D: Mobile Responsive Polish

Business Value: Coaches use tablets at gym; clients check programs on phones.

Deliverables:

1. Collapsible sidebar (hamburger on <768px)
2. Bottom tab nav for mobile (Dashboard | Session | Programs | Profile)
3. Assessment forms stack to single column

Generated by Kimi.ai

4. 3D body map touch controls optimized
5. Touch-friendly joint selection modal

Files to Modify:

- `css/styles.css` — add responsive breakpoints
- `index.html` — add mobile nav, data-responsive attributes
- `js/bodyMap3D.js` — enhance touch handlers (add pinch-zoom)

CRITICAL CONSTRAINTS (DO NOT VIOLATE)

Rule	Penalty for Violation
NEVER modify `scoring.js`	Breaks clinical validation
NEVER modify `gaitEngine.js`	Breaks gait phase logic
NEVER modify `bodyMap3D.js` core	Breaks 3D rendering
NEVER change Movement = ROM × Control × Force × Neurology formula	Breaks entire platform
ALWAYS use existing CSS variables	Maintains visual consistency
ALWAYS maintain Supabase RLS policies	Security breach risk
ALWAYS prefix new-session inputs with `ns-`	Form reader compatibility

---

DATABASE SCHEMA (Phase 3 Additions)

Community Tables

```sql
-- Coach peer messaging
CREATE TABLE coach_messages (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  sender_id uuid REFERENCES auth.users(id) ON DELETE CASCADE,
  receiver_id uuid REFERENCES auth.users(id) ON DELETE CASCADE,
  content text NOT NULL,
  created_at timestamptz DEFAULT now(),
  read_at timestamptz,
  CONSTRAINT no_self_message CHECK (sender_id != receiver_id)
);
```

```sql
-- Coach groups (specialty circles)
CREATE TABLE coach_groups (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  description text,
  created_by uuid REFERENCES auth.users(id),
  created_at timestamptz DEFAULT now()
);

CREATE TABLE coach_group_members (
  group_id uuid REFERENCES coach_groups(id) ON DELETE CASCADE,
  coach_id uuid REFERENCES auth.users(id) ON DELETE CASCADE,
  joined_at timestamptz DEFAULT now(),
  PRIMARY KEY (group_id, coach_id)
);

-- Client referrals between coaches
CREATE TABLE client_referrals (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  from_coach_id uuid REFERENCES auth.users(id),
  to_coach_id uuid REFERENCES auth.users(id),
  client_id uuid REFERENCES profiles(id),
  status text DEFAULT 'pending' CHECK (status IN ('pending','accepted','declined','completed')),
  notes text,
  created_at timestamptz DEFAULT now(),
  responded_at timestamptz
);

-- Case study sharing (anonymized)
CREATE TABLE case_shares (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  coach_id uuid REFERENCES auth.users(id),
  title text NOT NULL,
  description text,
  anonymized_data jsonb, -- stores assessment scores without PII
  tags text[],
  created_at timestamptz DEFAULT now()
);

-- Client community posts
CREATE TABLE client_posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id uuid REFERENCES profiles(id) ON DELETE CASCADE,
  content text NOT NULL,
  type text DEFAULT 'progress' CHECK (type IN ('progress','question','support','milestone')),
  created_at timestamptz DEFAULT now()
);

CREATE TABLE client_comments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  post_id uuid REFERENCES client_posts(id) ON DELETE CASCADE,
  author_id uuid REFERENCES auth.users(id),
  author_role text DEFAULT 'client' CHECK (author_role IN ('client','coach','admin')),
  content text NOT NULL,
  created_at timestamptz DEFAULT now()
);

-- Client support groups
CREATE TABLE client_groups (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  description text,
  focus_area text, -- e.g. "Lower Back Pain", "Post-Surgery Rehab"
  created_at timestamptz DEFAULT now()
);

CREATE TABLE client_group_members (
  group_id uuid REFERENCES client_groups(id) ON DELETE CASCADE,
  client_id uuid REFERENCES profiles(id) ON DELETE CASCADE,
  joined_at timestamptz DEFAULT now(),
  PRIMARY KEY (group_id, client_id)
);

-- Privacy settings per user
CREATE TABLE privacy_settings (
  user_id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  share_progress boolean DEFAULT false,
  share_posts boolean DEFAULT true,
  allow_comments boolean DEFAULT true,
  visible_to text DEFAULT 'coach_only' CHECK (visible_to IN ('public','coach_only','private'))
);
```

Exercise Library Tables

```
CREATE TABLE exercises (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  category text CHECK (category IN ('Rehab','Mobility','Strength','Neurology','Breathing')),
  phase text CHECK (phase IN ('Phase 1','Phase 2','Phase 3')),
  video_url text,
  thumbnail_url text,
  cues text,
  common_errors text,
  progressions text,
  regressions text,
  tags text[],
  target_joints text[], -- e.g. ['hip_ir','ankle_df']
  created_by uuid REFERENCES auth.users(id),
  created_at timestamptz DEFAULT now()
);
```

```sql
CREATE TABLE exercise_playlists (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  youtube_playlist_id text NOT NULL,
  name text,
  phase_mapping text CHECK (phase_mapping IN ('Phase 1','Phase 2','Phase 3','All')),
  auto_sync boolean DEFAULT false,
  last_synced_at timestamptz,
  created_at timestamptz DEFAULT now()
);
```

Progress Tracking Tables

```sql
CREATE TABLE progress_snapshots (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id uuid REFERENCES profiles(id) ON DELETE CASCADE,
  assessment_id uuid REFERENCES assessments(id),
  session_date date DEFAULT CURRENT_DATE,
  rom_score numeric(5,1),
  control_score numeric(5,1),
  force_score numeric(5,1),
  neurology_score numeric(5,1),
  composite_score numeric(5,1),
  phase text,
  created_at timestamptz DEFAULT now()
);
```

RLS POLICIES (Security Layer)

```sql
-- Coach messages: coaches can see their own conversations
CREATE POLICY "coaches_see_own_messages" ON coach_messages
  FOR ALL USING (auth.uid() IN (sender_id, receiver_id));

-- Client referrals: involved coaches only
CREATE POLICY "referral_participants" ON client_referrals
  FOR ALL USING (auth.uid() IN (from_coach_id, to_coach_id) OR Auth.is_admin());

-- Case shares: all coaches can read, only author can edit
CREATE POLICY "case_shares_read_all_coaches" ON case_shares
  FOR SELECT USING (Auth.is_coach_or_admin());
CREATE POLICY "case_shares_edit_author" ON case_shares
  FOR ALL USING (auth.uid() = coach_id);

-- Client posts: visibility controlled by privacy_settings
CREATE POLICY "client_posts_visibility" ON client_posts
  FOR SELECT USING (
    auth.uid() = client_id OR
    Auth.is_admin() OR
    EXISTS (
      SELECT 1 FROM privacy_settings ps
```

```
      WHERE ps.user_id = client_posts.client_id
      AND (ps.visible_to = 'public' OR
           (ps.visible_to = 'coach_only' AND Auth.is_coach()))
    )
  );

-- Comments: authors + post owners
CREATE POLICY "comment_access" ON client_comments
  FOR ALL USING (
    auth.uid() = author_id OR
    auth.uid() IN (SELECT client_id FROM client_posts WHERE id = post_id)
  );
```

IMPLEMENTATION ORDER (Smart Dependency Chain)

```
WEEK 1: Foundation
├── Day 1-2: Run SQL migrations, create empty JS files with TODO stubs
├── Day 3-4: Build Exercise Library tables + basic CRUD
├── Day 5: Connect ProgramGenerator to exercise library (replace hardcoded strings)

WEEK 2: Community Core
├── Day 1-2: Coach messaging (simplest community feature, builds patterns)
├── Day 3-4: Client posts + comments (extends messaging patterns)
├── Day 5: Privacy settings + visibility controls

WEEK 3: Community Advanced
├── Day 1-2: Coach groups + case sharing
├── Day 3-4: Client referrals workflow (most complex community feature)
├── Day 5: Support groups + moderation tools

WEEK 4: Visualization + Polish
├── Day 1-2: Progress snapshots table + chart.js integration
├── Day 3: Dashboard sparklines + client progress page
├── Day 4: Mobile responsive pass (sidebar, bottom nav, form stacking)
├── Day 5: Final integration testing + bug fixes
```

FILE CREATION CHECKLIST

Mark each [ ] as [x] when complete:

New Files

- `js/community.js` — Coach messaging, referrals, case shares
- `js/communityUI.js` — DOM rendering for all community features
- `js/exerciseLibrary.js` — YouTube API, exercise CRUD, search
- `js/exerciseUI.js` — Grid, video embed, “Add to Program”
- `js/charts.js` — Chart.js configuration for AST9 data

Generated by Kimi.ai

- `js/progressReport.js` — PDF generation

Modified Files (Safe to Edit)

- `index.html` — Add community nav items, exercise library section, chart containers, mobile nav
- `dashboard.js` — Add community notification counts, chart init calls, referral stats
- `programGenerator.js` — Replace hardcoded exercise strings with `ExerciseLibrary.lookup()`
- `css/styles.css` — Add responsive breakpoints, community card styles, video player styles

Untouched Files (DO NOT MODIFY)

- `scoring.js` — LOCKED
- `gaitEngine.js` — LOCKED
- `bodyMap3D.js` — LOCKED (except touch handler enhancement)
- `auth.js` — UNTOUCHED
- `clients.js` — UNTOUCHED
- `subscriptions.js` — UNTOUCHED
- `supabaseClient.js` — UNTOUCHED

TESTING CHECKLIST

Before declaring Phase 3 complete, verify:

- New session flow still works end-to-end (no regression)
- Scoring engine produces identical results to Phase 2
- Gait analysis renders same deficits for same inputs
- 3D body map colors match pain inputs
- Program generator respects phase gating (pain → P1 lock)
- Community: Coach A can message Coach B, not Coach C
- Community: Client privacy settings hide posts correctly
- Exercise library: Video embeds load, "Add to Program" inserts correct exercise
- Charts: Score history shows correct trend line
- Mobile: Sidebar collapses, forms stack, bottom nav works
- All new tables have RLS policies enforced

HANDOFF NOTES

If you are a developer picking up this prompt:

1. Start by reading the existing `index.html` to understand the DOM structure
2. Read `dashboard.js` to understand the app shell pattern
3. Run the SQL migrations FIRST — everything else depends on the schema
4. Build Workstream A (Community) before Workstream B (Exercise Library) — community is higher business value
5. When modifying `programGenerator.js`, keep the RULES array intact — only change how exercises are resolved from strings to library objects

Generated by Kimi.ai

6. Test on mobile early (Day 3 of Week 1) — responsive issues are harder to fix at the end

If you are Claude Code: Use `tree` to see file structure, read existing JS files for patterns, then implement workstream by workstream.

Generated by Kimi.ai